

Восстановление изображения сортировкой с полезной нагрузкой

Цель работы: Реализовать заданный алгоритм сортировки самостоятельно и применить его к сортировке записей вида «ключ — полезная нагрузка». Исследовать свойство устойчивости сортировки и его наблюдаемые последствия. Измерить фактическую скорость роста времени работы и сопоставить её с теоретической оценкой.

Теоретические сведения

Сортировка записей с полезной нагрузкой

В учебных примерах сортируют массив чисел. На практике сортируют записи: у каждой записи есть ключ, по которому идёт сравнение, и полезная нагрузка — данные, которые сравнению не подлежат, но обязаны переехать вместе со своим ключом.

В настоящей работе ключом является целое число из текстового файла, а полезной нагрузкой — пиксель изображения. Массивы ключей и пикселей имеют одинаковую длину и связаны по индексу: ключ с номером i описывает пиксель с номером i . Любая перестановка, выполняемая алгоритмом над массивом ключей, должна одновременно выполняться над массивом пикселей.

Это даёт удобное свойство: правильность работы видна глазом. Если перестановка применена верно, из перемешанного изображения собирается исходное. Если где-то ключи и пиксели разошлись, на экране будет шум.

Устойчивость сортировки

Сортировка называется устойчивой, если записи с одинаковыми ключами сохраняют в результате тот относительный порядок, в котором они располагались до сортировки. Неустойчивая сортировка вправе переставить их произвольно: формально результат отсортирован, но исходный порядок равных элементов утрачен.

Пока ключи различны, свойство никак себя не проявляет. Оно становится существенным ровно тогда, когда в данных есть повторы.

Обратите внимание: в выданном вам файле ключей числа повторяются. Что именно определяет порядок пикселей с одинаковым ключом — вопрос, на который придётся ответить в отчёте.

Устойчивость не является внутренним свойством задачи — это свойство конкретного алгоритма и конкретной его реализации. Некоторые алгоритмы устойчивы по своей природе, некоторые неустойчивы, а некоторые можно сделать устойчивыми, если аккуратно выбрать условие сравнения.

Алгоритм	Устойчивость	Причина
Вставками	устойчивая	Элемент сдвигается только мимо строго больших
Слиянием	устойчивая	При равенстве берётся элемент из левой половины
Шелла	неустойчивая	Сдвиги на большие расстояния меняют порядок равных
Быстрая	неустойчивая	Разделение переносит элементы через весь диапазон
Пирамидальная	неустойчивая	Просеивание не сохраняет порядок равных
Поразрядная	устойчивая	Каждый проход — устойчивая сортировка по одному разряду, от младшего к старшему

Оценки сложности

Алгоритм	Средний случай	Худший случай	Дополнительная память
Вставками	$O(n^2)$	$O(n^2)$	$O(1)$
Шелла	зависит от шагов	зависит от шагов	$O(1)$
Быстрая	$O(n \log n)$	$O(n^2)$	$O(\log n)$ на стек
Слиянием	$O(n \log n)$	$O(n \log n)$	$O(n)$
Слиянием (естественная)	$O(n \log n)$	$O(n \log n)$	$O(n)$
Пирамидальная	$O(n \log n)$	$O(n \log n)$	$O(1)$
Поразрядная	$O(d \cdot (n + k))$	$O(d \cdot (n + k))$	$O(n + k)$

Здесь n — число элементов, k — размер диапазона значений ключа, d — число разрядов. Обратите внимание, что для сортировок, не основанных на сравнениях, оценка зависит не только от n , и это придётся учесть при планировании замеров.

Формат BMP

Работа ведётся с 24-битным несжатым BMP. Файл состоит из заголовка файла (14 байт), информационного заголовка (40 байт) и массива пикселей. Все многобайтовые числа записаны в порядке от младшего байта к старшему.

Смещение	Размер	Поле	Значение
0	2	bfType	Сигнатура, всегда символы BM
2	4	bfSize	Полный размер файла в байтах
10	4	bfOffBits	Смещение начала пиксельных данных
14	4	biSize	Размер информационного заголовка, обычно 40
18	4	biWidth	Ширина в пикселях
22	4	biHeight	Высота; отрицательная означает строки сверху вниз
28	2	biBitCount	Бит на пиксель, в работе всегда 24
30	4	biCompression	Способ сжатия, в работе всегда 0

Три особенности формата, на которых чаще всего ошибаются:

- Порядок байтов в пикселе — синий, зелёный, красный, а не привычный RGB.
- Строки хранятся снизу вверх: первая строка пиксельных данных соответствует нижней строке изображения (если `biHeight` положительна).
- Длина строки в байтах дополняется до кратности четырём. Для ширины w длина строки равна $((w \cdot 3 + 3) / 4) \cdot 4$ с целочисленным делением. При ширине 200 дополнения нет, при ширине 201 в конце каждой строки лежит один лишний байт, который к изображению не относится и в сортировке участвовать не должен.
- Пиксельные данные не обязательно начинаются с байта 54. Поле `bfOffBits` — не константа, его нужно прочесть из заголовка и использовать как есть: информационный заголовок на практике бывает длиннее 40 байт (встречаются расширенные варианты), и тогда данные начинаются позже.

Нумерация позиций в файле ключей ведётся в порядке хранения пикселей в файле, то есть подряд от начала пиксельных данных, с пропуском байтов выравнивания. Если вы читаете файл

последовательно от начала пиксельных данных, в том порядке, в котором они лежат в файле, ваши номера совпадут с номерами в задании.

Условие задачи

Вам выдаются два файла: изображение с переставленными пикселями и текстовый файл с ключами. Требуется восстановить исходное изображение, отсортировав ключи выданным вам алгоритмом и применив ту же последовательность перестановок к пикселям.

Формат входных данных

Файл ключей: в первой строке одно целое число N — количество ключей. Далее N строк, в каждой одно целое число. Значение N совпадает с числом пикселей в изображении.

```
30000
1873
412
1873
95
...
```

Базовые требования к программе

1. Чтение BMP реализовано самостоятельно: разбор заголовков, учёт выравнивания строк, извлечение массива пикселей. Библиотеки работы с изображениями не используются.
2. Алгоритм сортировки реализован самостоятельно согласно варианту. Библиотечные сортировки запрещены; их разрешается использовать только для проверки собственного результата и в отчёте это должно быть оговорено.
3. Перестановки применяются к ключам и пикселям синхронно.
4. Восстановленное изображение записывается в корректный BMP-файл, открываемый любым просмотрщиком.
5. Программа запускается из командной строки с фиксированными параметрами (см. ниже) и не требует диалога с пользователем.
6. Программа не завершается аварийно ни на каком входном файле, включая пустой, усечённый и не являющийся BMP.
7. Предусмотрен режим замера времени, выводящий время сортировки в миллисекундах отдельно от времени чтения и записи файлов.

Формат запуска фиксирован, поскольку проверка автоматическая:

```
restore --bmp shuffled.bmp --keys keys.txt --out restored.bmp
restore --bmp shuffled.bmp --keys keys.txt --out restored.bmp --bench 7
```

Ключ `--bench` задаёт число повторов замера; программа выводит медиану времени сортировки. Чтение файлов при этом выполняется один раз, а сортировка запускается на копии исходных массивов.

Проверка результата

Помимо визуального контроля используйте выданную утилиту проверки. Она сравнивает ваш результат с эталоном и различает две ситуации: перестановка применена неверно и перестановка верна, но нарушен порядок элементов с одинаковыми ключами.

```
bmpshuffle check -orig source.bmp -got restored.bmp -group 8
```

Варианты

Номер варианта, а также параметры генерации выданных вам файлов сообщает преподаватель. Реализовать нужно именно указанный алгоритм; замена его на другой означает невыполнение работы.

№	Алгоритм	На что обратить внимание
1	Сортировка вставками	Поведение на почти отсортированных данных
2	Сортировка бинарными вставками	Число сравнений против числа перемещений
3	Сортировка Шелла, шаги Хиббарда	Зависимость от последовательности шагов
4	Сортировка Шелла, шаги Седжвика	Сравнение с вариантом 3
5	Быстрая, опорный — первый элемент	Худший случай достигается легко
6	Быстрая, опорный — медиана трёх	Насколько это спасает от худшего случая
7	Быстрая, опорный случайный	Что именно гарантирует рандомизация
8	Сортировка слиянием (рекурсивная)	Расход дополнительной памяти
9	Сортировка слиянием (естественная)	Лучший случай на уже упорядоченных участках
10	Пирамидальная сортировка	Стоимость построения кучи
11	Поразрядная сортировка, LSD, по десятичным разрядам	Число проходов растёт с диапазоном ключей гораздо медленнее самого диапазона

Дополнительное задание (повышенный уровень)

Выбрать не менее двух пунктов.

- Анимация процесса сортировки: сохранение промежуточных состояний массива в виде последовательности кадров или видеофайла. Достаточно сохранять кадр раз в K операций, значение K подобрать так, чтобы ролик длился десятки секунд.
- Худший случай: построить входные данные, на которых ваш алгоритм показывает наихудшее поведение, и подтвердить деградацию замером в сравнении со случайными данными того же размера. Объяснить, из какого свойства алгоритма возникает деградация.
- Исправление устойчивости: если ваш алгоритм неустойчив, сделать его устойчивым, не меняя самого алгоритма, — за счёт расширения ключа. Предъявить точное совпадение с эталоном после исправления и оценить, во что обошлось исправление по памяти и времени.
- Устойчивость к повреждённым данным: обработать усечённый файл, файл с неверной сигнатурой, значение `biBitCount`, отличное от 24, и файл ключей, длина которого не совпадает с числом пикселей. Для каждого случая — внятное сообщение с указанием причины.
- Сравнение с библиотечной сортировкой: измерить обе на одних данных и объяснить разрыв. Ответ «библиотека написана лучше» не принимается: нужно назвать конкретные причины.
- Внешняя сортировка: обработать изображение, которое не помещается в оперативную память целиком, разбиением на блоки с последующим слиянием.

Требования к отчёту

Отчёт оформляется по общему шаблону курса и содержит четыре обязательных раздела.

1. Асимптотика вашего алгоритма по времени и памяти с обоснованием, а не со ссылкой на таблицу из методички. Для рекурсивных алгоритмов — рекуррентное соотношение и его решение.
2. Замеры по протоколу курса: не менее шести размеров входа, растущих удвоением, не менее семи повторов, медиана, указание разброса. График и проверка гипотезы роста нормировкой $t(n)/f(n)$.
3. Худший случай: механизм возникновения, конструкция входа, замер.
4. Масштабирование: оценка времени и памяти при увеличении входа в тысячу раз и указание того, что откажет первым.

Отдельным пунктом в разделе выводов — ответ на вопрос о повторяющихся ключах: что определило порядок пикселей с одинаковым ключом в вашей реализации и совпал ли ваш результат с эталоном побайтово. Если не совпал, приложите увеличенный фрагмент изображения с видимыми искажениями.

Разные размеры изображений дают естественную шкалу n для замеров; набор изображений разного размера выдаётся вместе с вариантом.

Контрольные вопросы

1. Что такое устойчивость сортировки и в каких задачах она существенна?
2. Ваш алгоритм устойчив? Докажите это рассуждением о том, при каком условии происходит перестановка равных элементов.
3. Можно ли сделать любую сортировку устойчивой? Какой ценой?
4. Почему при сортировке записей время работы может заметно отличаться от времени сортировки одних только ключей?
5. Что произойдёт, если применять перестановки к массиву ключей, но забыть применить их к пикселям? Как это будет выглядеть на изображении?
6. Как выглядят входные данные, на которых ваш алгоритм работает быстрее всего? Медленнее всего?
7. Почему длина строки в BMP дополняется до кратности четырём и что произойдёт, если это игнорировать?
8. Существует ли алгоритм сортировки сравнениями со сложностью лучше, чем $O(n \log n)$ в худшем случае? Обоснуйте ответ.
9. Поразрядная сортировка работает за $O(d \cdot (n + k))$, фактически почти линейно по n . Почему она не вытеснила все сравнивающие сортировки?